

Дорошенко ис233 7labe

```
1) fun main() {  
    println("Введите первое число:")  
    val number1 = readLine()?.toDoubleOrNull() ?: return  
    println("Некорректный ввод")  
  
    println("Введите второе число:")  
    val number2 = readLine()?.toDoubleOrNull() ?: return  
    println("Некорректный ввод")  
  
    println("Выберите операцию (+, -, *, /):")  
    val operation = readLine()  
  
    val result = when (operation) {  
        "+" -> number1 + number2  
        "-" -> number1 - number2  
        "*" -> number1 * number2  
        "/" -> {  
            if (number2 != 0.0) number1 / number2 else "На ноль делить  
нельзя"  
        }  
        else -> "Некорректная операция"  
    }  
  
    println("Результат: $result")  
}
```

The screenshot shows an IDE window with the Kotlin code from the previous block. The code is in a file named 'pract 7.1'. The IDE interface includes a file explorer on the left, a code editor in the center, and a run console at the bottom. The run console shows the output of the program: 'Введите первое число: 22', 'Введите второе число: 12', 'Выберите операцию (+, -, *, /): -', and 'Результат: 10.0'. The process finished with exit code 0. There are also some warning messages in the console related to the Java runtime environment.

```

2) fun main() {
    println("Введите слово:")
    val input = readLine()?.trim() ?: return println("Некорректный ввод")

    val isPalindrome = input.equals(input.reversed(), ignoreCase = true)

    if (isPalindrome) {
        println("$input является палиндромом.")
    } else {
        println("$input не является палиндромом.")
    }
}

```

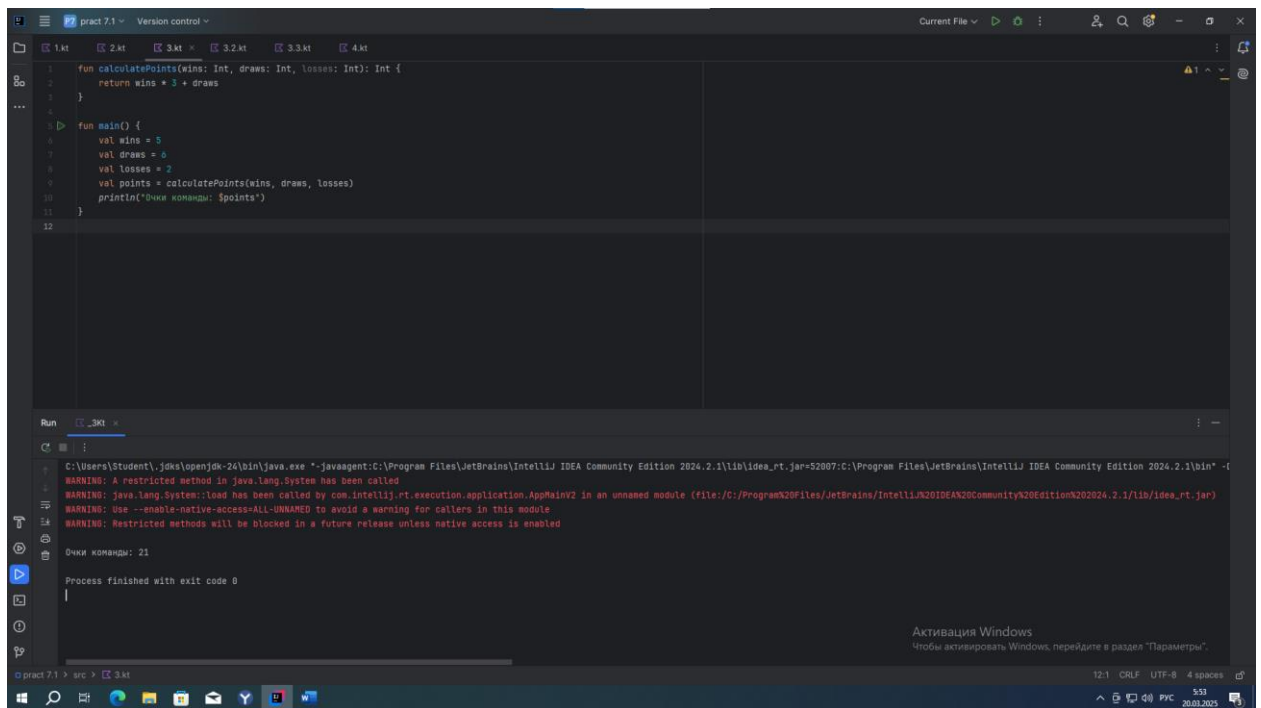
The screenshot shows the IntelliJ IDEA interface. The top editor displays the Kotlin code for checking a palindrome. The bottom 'Run' window shows the execution output: 'Введите слово:' followed by 'слово' (input) and 'слово не является палиндромом.' (output). The process finished with exit code 0. A Windows activation watermark is visible in the bottom right corner.

```

3) fun calculatePoints(wins: Int, draws: Int, losses: Int): Int {
    return wins * 3 + draws
}

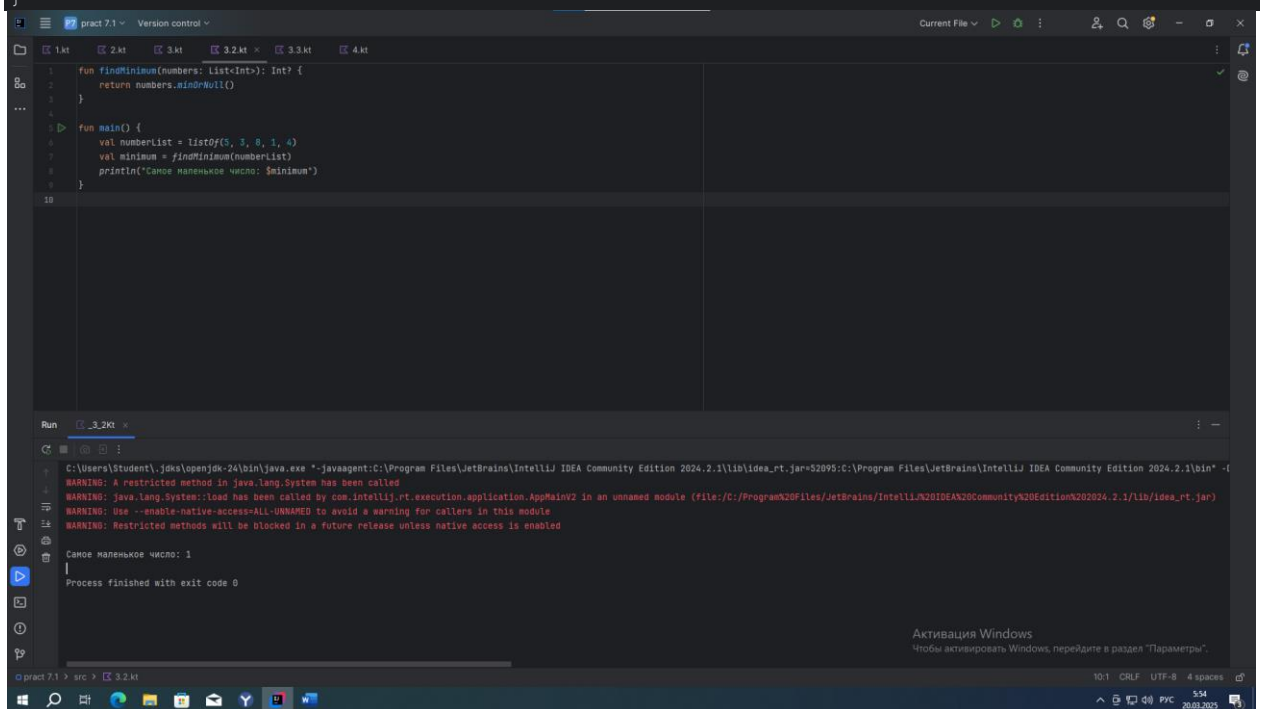
fun main() {
    val wins = 5
    val draws = 6
    val losses = 2
    val points = calculatePoints(wins, draws, losses)
    println("Очки команды: $points")
}

```



```
3.2) fun findMinimum(numbers: List<Int>): Int? {
    return numbers.minOrNull()
}

fun main() {
    val numberList = listOf(5, 3, 8, 1, 4)
    val minimum = findMinimum(numberList)
    println("Самое маленькое число: $minimum")
}
```

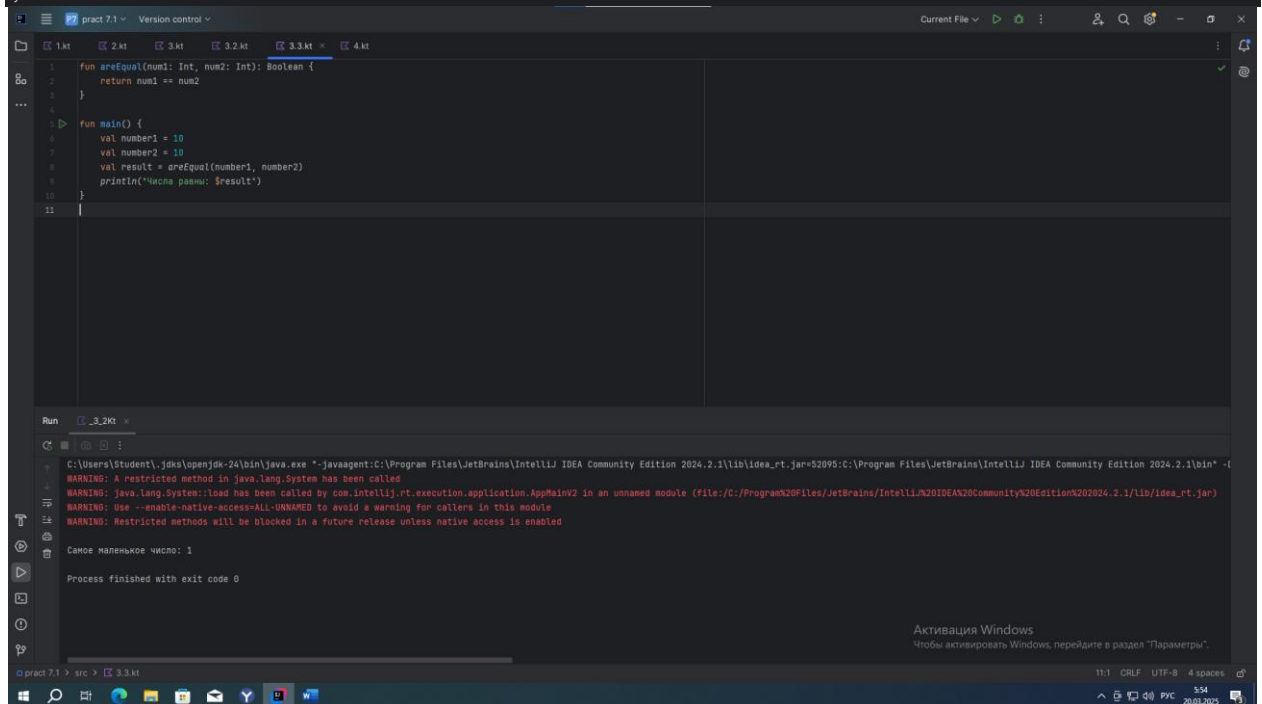


```

3.3) fun areEqual(num1: Int, num2: Int): Boolean {
    return num1 == num2
}

fun main() {
    val number1 = 10
    val number2 = 10
    val result = areEqual(number1, number2)
    println("Числа равны: $result")
}

```



```

4) import kotlin.random.Random

fun main() {
    var playerScore = 0
    var dealerScore = 0
    val deck = (1..11).toList() + (1..10).toList().repeat(4)

    while (true) {
        playerScore += drawCard(deck)
        println("Ваш счёт: $playerScore")

        if (playerScore > 21) {
            println("Вы перебрали! Игра окончена.")
            break
        }

        println("Хотите взять ещё карту? (y/n)")
        val input = readLine()
        if (input != "y") break
    }

    if (playerScore <= 21) {
        while (dealerScore < 17) {
            dealerScore += drawCard(deck)
        }
    }
}

```

```

println("Счёт дилера: $dealerScore")

when {
    dealerScore > 21 -> println("Дилер перебрал! Вы выиграли!")
    dealerScore == playerScore -> println("Ничья!")
    dealerScore > playerScore -> println("Дилер выиграл!")
    else -> println("Вы выиграли!")
}

}

}

fun drawCard(deck: List<Int>): Int {
    val card = deck.random()
    println("Вы вытянули карту: $card")
    return card
}

fun List<Int>.repeat(times: Int): List<Int> {
    return this.flatMap { List(times) { it } }
}

```

The screenshot shows an IDE window with a Kotlin file named '4.kt'. The code defines a main function, a drawCard function, and a List.repeat extension function. The Run console shows the execution output, which matches the code logic: drawing cards until the dealer's score is 17, then announcing the dealer's win.

```

Run 4.kt
Вы вытянули карту: 11
Ваш счёт: 21
Хотите взять ещё карту? (y/n)
n
Вы вытянули карту: 6
Вы вытянули карту: 1
Вы вытянули карту: 0
Вы вытянули карту: 3
Вы вытянули карту: 3
Вы вытянули карту: 2
Вы вытянули карту: 2
Счёт дилера: 17
Вы выиграли!

```

At the bottom of the IDE, there is a Windows activation watermark: "Активация Windows. Чтобы активировать Windows, перейдите в раздел 'Параметры'."